1. Write a program in C/C++ to simulate the Banker's algorithm for deadlock avoidance. Consider at least 3 processes in the system, with 4 resource classes having at least one resource instance for each class. Assume the values for Available, Allocation, MAX, and request from a particular process from your side. Program must reflect two cases, where a safe sequence exists for one and safe sequence does not exist for another.

2. Write a C/**C++ program** to implement a simulator for variable partition memory allocation strategies. The simulator must read the required input data from a file named **alloc.dat**.

   The first line of the file contains:

- **Memory size** in KB.

   Each subsequent line contains a 3-tuple:

< process_id, arrival_time, block_size, duration>

   where:

   arrival_time = time at which the memory allocation request arrives.

   block_size = amount of memory requested in KB.

   duration = time for which the allocated block remains in use.
   (duration > 0)

The input terminates with the following line:

-1 -1 -1

Allocation Rules

1. All memory allocation requests should be maintained in a **waiting queue** until they can be successfully allocated.
2. If a request arrives at time **T** and is successfully allocated at time **T**, the allocated block must remain occupied for the specified duration.
3. The allocated memory becomes free only after:

   T + duration

4. Once a block becomes free, it should be made available for subsequent allocation requests.
5. If multiple requests are waiting, they should be considered according to their order in the waiting queue.
6. The simulator should maintain the status of allocated and free memory partitions throughout the execution.

Implement and simulate the following four **variable partition allocation strategies** independently:

1. **First Fit**
2. **Best Fit**
3. **Worst Fit**

Output

For **each allocation strategy**, report the following statistics **after every 10 allocation requests**:

1. **Percentage of successful allocation requests**

   Success Percentage = (Number of successfully allocated requests/ Total requests considered) *100

2. **External fragmentation** as a percentage of the overall memory.
3. **Internal fragmentation** as a percentage of the overall memory.